Python Programming Fundamentals

Python is an **interpreted, high-level scripting language** — widely used in **DevOps, Cybersecurity, and Automation** for scripting, infrastructure automation, and system integration.

- ☐ **A high-level**, **dynamically typed** programming language, used to instruct the computer line by line.
- ☐ No compilation needed — just write and run.

Python 3 Installation and Setup on Windows

1. Install **Python 3** on a Windows machine
2. Configure the **PATH environment variable** (if not done automatically)
3. **Verify** the installation
4. Configure **VS Code** or another IDE for Python development

---

Step 1: Check if Python is Already Installed

Many Windows 10/11 systems come with Python preinstalled.

To check, open **Command Prompt (CMD)** and type:

python --version

or

python3 --version

- ☐ If you see an output like `Python 3.x.x`, Python is already installed.
- ☐ If not, continue to the next step.

Step 2: Download Python 3 Installer

Go to the official Python website:
🔗 https://www.python.org/downloads/windows/

1. Click **"Download Python 3.x.x"**
2. A file named similar to `python-3.12.6-amd64.exe` will be downloaded

Step 3: Start Installation Process

1. Double-click the downloaded installer file to open it

2. **Important:** Check the box **"Add Python 3.x to PATH"**
   - This automatically configures the environment variable, allowing you to run Python directly from the terminal

3. Click **Install Now**

4. Wait for the installation to complete

5. Once done, you'll see a message like:
   ✅ **Setup was successful!**

Step 4: Verify the Installation

Open **Command Prompt** and type:

```
python --version
```

or

```
python3 --version
```

Expected output:

Python 3.12.6

To test the interpreter:

python

Then run:

print("Hello, Python!")

Output:

Hello, Python!

To exit the Python shell:

exit()

Step 5: Verify PIP (Python Package Manager)

PIP is Python's package manager, used to install external libraries.

Check if PIP is installed:

pip --version

Expected output example:

pip 24.0 from
C:\Users\YourName\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip
(python 3.12)

Step 6: Install Visual Studio Code (Optional but Recommended)

Download VS Code:
🔗 https://code.visualstudio.com/download

After installation:

1. Open **VS Code**
2. Go to **Extensions** → search for **Python**
3. Install the **official Microsoft Python extension**

Step 7: Keep Python and PIP Updated

To update **pip**:

python -m pip install --upgrade pip

If you have multiple Python versions installed, you can specify which version to use:

py -3.12 --version

Bonus Tip: Fix PATH Issues Manually

If `python` is not recognized, you may need to set the PATH manually.

Manual PATH Setup:

1. Right-click **This PC** → **Properties**
2. Click **Advanced system settings**
3. Click **Environment Variables**
4. Under “System Variables,” select **Path** → **Edit**
5. Add the following paths:

```
C:\Users\<YourUserName>\AppData\Local\Programs\Python\Python312\
C:\Users\<YourUserName>\AppData\Local\Programs\Python\Python312\Scripts\
```

6. Click **OK**, restart CMD, and verify again.

Final Verification

Run both commands to confirm setup:

python3 --version
pip3 --version

- ☐ If both return valid versions, your Python environment is ready to use

---

Python Setup & Environment For Linux/Ubuntu systems:

1. Install Python 3

sudo apt update
sudo apt install python3 python3-pip -y

Verification:

python3 --version
pip3 --version

- ☐ Python3 and pip3 should display their versions successfully.

---

3. Add Essential VS Code Extensions

Recommended Python Extension Pack

Extension	Purpose
Python	Core language support
Pylance	IntelliSense, code navigation
Python Indent	Proper indentation
Code Runner	Run code snippets directly
Python Environment Manager	Manage virtual environments
Black Formatter	Auto format Python code
Python Environment	Detect & switch Python interpreters

---

4. Configure Code Formatter (Black Formatter)

Install Black Formatter extension

Then configure the formatter inside VS Code **settings.json**:

```
"[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.formatOnSave": true
}
```

To format your code manually:
Press `Shift + Option + F` (macOS) or `Shift + Alt + F` (Windows/Linux)

Syntax

Key Syntax Rules

- **Indentation is mandatory** (no curly braces like Bash or PowerShell)
- **Case-sensitive** → `Name` ≠ `name`
- **Comments**
    - Single line → `#`
    - Multi-line → `""" comment """`

Your First Python Script

Run your first Python file

```
# This is my first Python program
print("Hello, Cyber Security Automation World!")
print("=== CYBER SECURITY SCANNER ===")
print("Starting system analysis...")
```

Explanation

- `print()` → displays text output on screen
- Text inside `"  "` is printed exactly as written
- `#` → comments ignored by interpreter

Run:

python3 hello.py

Output:

Hello, Cyber Security Automation World!

Variables

Variables are **containers** used to store data in memory.
Python automatically detects data types — no manual declaration required.

Example

```python
# create Variables
target_ip = "192.168.1.1"           # String (text)
scan_timeout = 30                   # Integer (number)
is_stealth_mode = True              # Boolean (True/False)
ports_to_scan = [22, 80, 443, 3389] # List (multiple values)
```

Naming Rules

- Start with a letter or _
- No spaces: `target_ip` , wrong-one– `target ip`
- Cannot start with a number
- Case-sensitive
- Avoid reserved keywords (`if`, `for`, etc.)

For example: `True` is a Boolean value — Python automatically detects variable type.

Data Types

`type()` → Built-in function that identifies the variable's data type.

Type	Example	Description
`str`	`"Nahid"`	String/Text
`int`	`20`	Integer
`float`	`20.5`	Decimal number
`bool`	`True` / `False`	Boolean logic
`list`	`[1, 2, 3]`	Ordered collection
`dict`	`{"name": "Nahid", "age": 28}`	Key-value pair

Operators

Operators perform calculations, comparisons, and logical evaluations.

Arithmetic Operators

Operator	Example	Description
`+`	`a + b`	Addition
`-`	`a - b`	Subtraction
`*`	`a * b`	Multiplication
`/`	`a / b`	Division
`%`	`a % b`	Modulus (remainder)
`**`	`a ** b`	Exponentiation
`//`	`a // b`	Floor Division

```python
# Operators for security calculations
print("=== SECURITY OPERATORS ===")

# Arithmetic Operators (Math)
failed_logins = 15
successful_logins = 85
total_logins = failed_logins + successful_logins
success_rate = (successful_logins / total_logins) * 100

print("Total Logins:", total_logins)
print("Success Rate:", success_rate, "%")
```

Comparison Operators

Operator	Example	Meaning
`==`	`a == b`	Equal
`!=`	`a != b`	Not Equal
`>`	`a > b`	Greater than
`<`	`a < b`	Less than
`>=`	`a >= b`	Greater or Equal
`<=`	`a <= b`	Less or Equal

```python
# Comparison Operators (Compare values)
max_failed_attempts = 5
current_attempts = 8

print("Current Attempts:", current_attempts)
print("Max Allowed:", max_failed_attempts)
print("Over Limit?", current_attempts > max_failed_attempts)
```

Logical Operators

Operator	Example	Description
`and`	`a > 5 and b < 10`	True if both true
`or`	`a > 5 or b < 10`	True if one is true
`not`	`not(a > 5)`	Negates the result

```python
# Logical Operators (AND/OR)
has_admin_access = True
is_business_hours = False

# AND operator - both must be True
can_access_sensitive = has_admin_access and is_business_hours
print("Access Sensitive Data?", can_access_sensitive)

# OR operator - any one can be True
can_access_basic = has_admin_access or is_business_hours
print("Access Basic Data?", can_access_basic)
```

Conditional Statements (if, elif, else)

Conditional statements allow **decision-making** in scripts.

```python
if condition:
    statement1
elif another_condition:
    statement2
else:
    statement3
```

Used in:

- File existence checks
- Login validation
- Server monitoring logic

```python
x = int(input("Enter a number: "))
if x > 0:
    print("It Is A Positive Number")
else:
    print("It Is Not A Positive Number")
```

Mini Practice Challenge

Task

Create `system_check.py` script:

1. Take system name as input
2. If system == `"server1"` → print “Server Active”
3. If system == `"backup"` → print “Backup Mode Active”
4. Else → print “Unknown System”

Bonus: Add timestamp

```
from datetime import datetime
print(datetime.now())
```

Summary of Learning Till Now

Topic	Key Takeaway
Syntax	Indentation defines structure
Variables	Dynamically typed
Data Types	Built-in types for text, numbers, collections
Operators	Arithmetic, Comparison, Logical
Conditionals	Used for decision-making

Collections — Lists, Dictionaries, Tuples, Sets

Why Collections Are Important

In real-world automation or security scripting, data is typically presented in groups — such as IP lists, usernames, file names, or JSON records.
Collections make it easy to **store, organize, filter, and manipulate** such grouped data in a clean and readable way.

A. Lists (`list`)

- ☐ A list is a data structure in programming that allows you to store multiple items in a single variable
- ☐ It maintains the order of elements (the sequence in which you add them)
- ☐ Lists are mutable, meaning you can change, add, or remove items after creation.
- ☐ You can access elements by index and modify them as needed.

Syntax Example:

```
ips = ["10.0.0.1", "10.0.0.2", "192.168.1.5"]
```

Common Operations

Method	Description
`append()`	Add element at the end
`insert()`	Add element at a specific index
`pop()`	Remove and return last element
`remove()`	Delete specific element
`ips[0:2]`	Slice list to get subset
`for ip in ips:`	Iterate through list

B. Dictionaries (`dict`)

A **dictionary** stores data as key-value pairs, allowing quick lookup by key — similar to JSON structure.

Example:

```
user = {"name": "xyz", "role": "devops", "email": "xyz@example.com"}
```

Common Operations

Operation	Example	Purpose
Access value	`user["role"]`	Retrieve data by key
Add/Update	`user["team"] = "platform"`	Modify existing or add new key
Safe access	`user.get("phone", "unknown")`	Prevents key errors
Iterate	`for k, v in user.items():`	Loop through pairs

```
# Create dictionaries
user_roles = {
    "admin": "Full access",
    "user": "Limited access",
    "guest": "Read-only access"
}

firewall_rules = {
    "ssh": {"port": 22, "action": "allow"},
    "http": {"port": 80, "action": "allow"},
    "telnet": {"port": 23, "action": "block"}
}

malware_db = {
    "trojan": "Steals data",
    "ransomware": "Encrypts files",
    "spyware": "Monitors activity"
}
```

C. Tuples (`tuple`)

A **tuple** is similar to a list, but it is **immutable** (cannot be changed).

Used for fixed data such as standard ports, constant settings, or version info.

Example:

```
standard_ports = (22, 80, 443)
```

Why Use Tuples?
✔️ Prevent accidental modification
✔️ Ideal for storing fixed or constant data

D. Sets (`set`)

A **set** is an unordered collection of **unique** items.
Perfect for filtering duplicates and performing fast membership checks.

Example:

```
blocked_ips = {"10.0.0.2", "192.168.1.5"}
```

Common Operations

Operation	Description
`add()`	Add a new element
`remove()`	Delete an element
`union()`	Combine two sets
`intersection()`	Find common elements
`difference()`	Exclude overlapping items

Example:

```
monitoring_ips = {"192.168.1.1", "192.168.1.2", "192.168.1.5"}
suspicious_ips = {"192.168.1.2", "192.168.1.3", "192.168.1.6"}

print("Common IPs:", monitoring_ips & suspicious_ips)
print("All Unique IPs:", monitoring_ips | suspicious_ips)
print("Only in Monitoring:", monitoring_ips - suspicious_ips)
```

Use Case:

- De-duplicating IP logs
- Identifying overlapping alerts
- Quick membership checks in intrusion detection

Match Case Statement

A. Match Case (Switch-like Control Flow)

Introduced in **Python 3.10+**, `match` is similar to a switch statement used for cleaner multi-condition handling.

Example:

```python
status = "running"

match status:
    case "running":
        print(" Service running")
    case "stopped":
        print(" Service stopped")
    case _:
        print(" Unknown status")
```

Notes:

- `_` acts as a wildcard or default case.
- Cleaner than multiple `if-elif-else` blocks.
- Used for structured data matching (e.g., JSON validation).

Loops — Repetition & Automation

Why Loops?

Used to **repeat tasks** — scanning ports, validating files, checking multiple servers, etc.

1. **For Loop**

Iterates over sequences like lists, tuples, or ranges.

Example 1:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

Mini Challenge

```
# Imagine you're managing a network security system
blocked_ips = ["192.168.1.100", "10.0.0.50", "172.16.0.25"]
new_threats = ["192.168.1.100", "203.0.113.15", "10.0.0.75"]

print("Current blocked IPs:", blocked_ips)

for threat in new_threats:
    if threat not in blocked_ips:
        blocked_ips.append(threat)
        print(f"NEW THREAT BLOCKED: {threat}")
    else:
        print(f"Already blocked: {threat}")

print("Updated blocked IPs:", blocked_ips)
```

- ☐ How many new threats will be actually blocked and why?
- ☐ What's the role of the for loop in this security context?
- ☐ Why is it important to check if threat not in blocked_ips?

Challenge 2: Debugging Task

```
ips = ["10.0.0.1", "10.0.0.2", "10.0.0.4"]
new_ips = ["192.168.1.1", "10.0.0.5"]

for new_ip not in ips:
    ips.append(new_ips)
print(ips)
```

2. **While Loop**

With the while loop, we can execute a set of statements as long as a condition is true.

```
count = 0
while count < 3:
    print("Count:", count)
    count += 1
```

Break Statement

Immediately exits the loop.

```
for fruit in ["apple", "banana", "cherry"]:
    if fruit == "banana":
        break
    print(fruit)
```

Continue Statement

Skips the current iteration, continues with next.

```
for fruit in ["apple", "banana", "cherry"]:
    if fruit == "banana":
        continue
    print(fruit)
```

Practice Challenge

Task:
Write a Python script that:

1. Stores a list of suspicious IPs
2. Checks if `"192.168.1.5"` exists
3. If found → print “Alert: IP under monitoring”
4. Else → print “Safe connection”

Functions

A function is a reusable block of code designed to perform a specific task.

- ➢ Functions help make your scripts **modular, organized, and easier to maintain.**
- ➢ Instead of writing the same logic multiple times, you define it once and call it anywhere.

Why Do We Use Functions?

Problem Without Function	Benefit With Function
Code duplication	Reuse logic anywhere in the script
Complex maintenance	Update code in one place only
Difficult debugging	Centralized logic makes issue tracking simpler
Low readability	Improves clarity and structure

Function Syntax

```
def function_name(parameters):
    # Function body
    return value
```

- ➔ `def` — defines the function
- ➔ `function_name` — unique identifier
- ➔ `parameters` — input values
- ➔ `return` — sends output back to the caller

Example — Basic Function

```
def greet_user():
    print("Welcome to Python Automation Training!")
```

Explanation:

- Defines a function named **greet_user**

- Executes when you **call** it:

greet_user()

Example 2 — Function with Parameters

```
def greet_user(name):
    print(f"Hello {name}, welcome to the training!")
```

Explanation:

- `name` → acts as a **parameter**
- Value dynamically received during call:

greet_user("XYZ")

- ☐ Uses **f-string formatting** to inject variable values dynamically.

Example— Function with Return Value

```
def add_numbers(a, b):
    return a + b
```

Explanation:

- Function takes two parameters (`a`, `b`)
- `return` sends result back to caller

Execution:

```
result = add_numbers(10, 20)
print("Sum is:", result)
```

`return` → makes your function reusable in calculations, reports, and automation logic.

Hands-On Lab : Counting Log Files

Objective: Create a Python function to count how many `.log` files exist in a directory.

```
import os

def count_log_files(directory):

    files = os.listdir(directory)

    log_files = [f for f in files if f.endswith(".log")]

    return len(log_files)


path = "/Users/nahid/Training/Scripting-training/Python/"

print("Total .log files:", count_log_files(path))
```

Explanation:

- `os.listdir()` → fetches all files in a directory
- `.endswith(".log")` → filters only `.log` files
- `len()` → returns the count

File I/O Operations

File I/O (Input/Output) in Python allows programs to interact with files — reading, writing, or modifying data stored on disk.

➢ Core to **Automation, System Administration, and Cybersecurity Operations**
➢ Commonly used for:
- Log analysis
- Configuration updates
- Report generation
- Data exchange (CSV/JSON/YAML formats)

1️⃣ Reading & Writing Plain Text Files

Example — Writing to a Log File

```
with open("activity.log", "w") as f:

    f.write("Log file created successfully.\n")
```

```
    f.write("System scan started...\n")
```

- ➔ `with open()` — ensures file is automatically closed
- ➔ `"w"` — write mode, overwrites existing file

Example — Appending Log Data

```
with open("activity.log", "a") as f:
    f.write("Scan completed successfully.\n")
```

- ➔ `"a"` — append mode, preserves previous content

Example — Reading File Content

```
with open("activity.log", "r") as f:
    content = f.read()
    print(content)
```

- ➔ `"r"` — read mode
- ➔ Ideal for parsing system logs or configuration files

Log Append Automation

Objective: Build a system log writer for monitoring events.

```
logfile = "system.log"


def write_log(msg):
    with open(logfile, "a") as f:
        f.write(f"{msg}\n")
```

```
write_log("System started successfully.")

write_log("Security scan completed.")
```

- Each call appends a new log line
- Perfect for **continuous system monitoring automation**

2️⃣ Working with CSV Files

Writing CSV File

```
import csv


data = [
    ["Name", "Role"],
    ["XYZ", "DevOps Engineer"],
    ["Rafi", "Security Analyst"]
]


with open("users.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

Note: newline="" means it removes extra space and unwanted blank rows from the CSV file

Reading CSV File

```
with open("users.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

- **Use Case:** Exporting system scan results, inventory reports, or compliance data

---

3️⃣ Working with JSON Files

```
import json
data = {
    "Server": "Prod-1",
    "Status": "Running",
    "IP": "192.168.1.10"
}

with open("server.json", "w") as f:
    json.dump(data, f, indent=4)
```

Reading JSON

```
with open("server.json", "r") as f:
    info = json.load(f)
    print(info["Server"])
```

Use Case: JSON is the backbone of **API data exchange, configuration management, and cloud automation tools (e.g., Terraform, Ansible)**

Sample JSON data:

```
data = {
  "name": "John",
  "age": 30,
  "married": True,
  "divorced": False,
  "children": ("Ann","Billy"),
  "pets": None,
  "cars": [
    {"model": "BMW 230", "mpg": 27.5},
    {"model": "Ford Edge", "mpg": 24.1}
  ]
}
```

4 Working with YAML Files

YAML (Yet Another Markup Language) is lightweight and human-readable — heavily used in DevOps pipelines and Kubernetes.

```
import yaml
```

```
data = {
    "app": "web-server",
    "port": 8080,
    "status": "active"
}

with open("config.yaml", "w") as f:
    yaml.dump(data, f)
```

To import and work with YAML files in Python, the PyYAML library is required. This library is not part of the Python standard library, so it must be installed first.

pip3 install pyyaml

Reading YAML

```
with open("config.yaml", "r") as f:
    config = yaml.safe_load(f)
    print(config["app"])
```

Use Case: Used in **CI/CD pipelines (GitHub Actions, Jenkins)** and **Kubernetes manifests**

Error Handling in File I/O

Example — File Not Found Handling

```
try:
    with open("missing.log", "r") as f:
```

```
    data = f.read()
except FileNotFoundError as e:
    print("Error: File not found!", e)
```

- ☐ Prevents script crashes due to missing files
- ☐ Crucial in **production-grade automation and security scripts**

Hands-On Lab: Error-Handled File Reader

Goal: Safely read a file and log failure messages.

```
try:
    with open("logs/activity.log", "r") as f:
        print(f.read())
except Exception as e:
    with open("error.log", "a") as err:
        err.write(f"{e}\n")
```

Implements **structured logging** and **graceful error handling**

Summary

Concept	Key Idea	Real-Life Use Case
Functions	Reusable logic block	Log scanner, API automation
File I/O	Read/write local files	Log storage, configuration management
JSON	Structured data format	API responses, cloud automation
YAML	Human-readable configs	CI/CD pipelines, Kubernetes manifests
CSV	Tabular data format	Report generation, data export/import

---

Working with OS-level Tasks in Python

(os, subprocess, shutil)

1️⃣ OS Module

Purpose: The `os` module allows Python scripts to communicate with the operating system.

Used for:

- Automating file and directory operations
- Reading environment variables
- Accessing system-level information

Example: OS Module Basics

```
import os

print("=== OS MODULE BASIC OPERATIONS ===")

# Current directory
print("Current Directory:", os.getcwd())
```

List files and directories

```
print("\n2. Files and Directories:")
items = os.listdir(".")
for item in items:
    if os.path.isfile(item):
        print(f" {item} (File)")
    elif os.path.isdir(item):
        print(f" {item}/ (Directory)")
```

# Create directory
os.makedirs("security_scans", exist_ok=True)
print(" Created: security_scans")

# Environment variables
print("Current user:", os.environ.get("USER", "Unknown"))
print("Home directory:", os.environ.get("HOME", "Unknown"))

Use: Automatically manage log directories, detect environment paths, or prepare security scan folders.

Real-Life Automation Use Case

Scenario	Manual Approach	Python Automation
Creating log directories	Create manually each time	Automatically create if not found
Collecting system info	Run commands one by one	Single Python script gathers all
Checking file existence	Manually verify	Script automatically validates

2 Subprocess Module

Purpose

Enables your Python program to **execute system commands**, interact with the shell, and capture command output.

Used for:

- Running terminal commands (`ping`, `netstat`, etc.)
- Gathering system information
- Automating security or network checks

Example: Running System Commands

```
import subprocess, platform

print("=== SUBPROCESS MODULE ===")

# Run basic command
result = subprocess.run(["whoami"], capture_output=True, text=True)
print("Current user:", result.stdout.strip())

# OS detection
system_os = platform.system()
command = ["systeminfo"] if system_os == "Windows" else ["uname", "-a"]

# Execute and capture output
result = subprocess.run(command, capture_output=True, text=True)
print("System Info:", result.stdout[:200])
```

Example: Automated network scanning, OS detection, process monitoring, or server inventory scripts.

Example: Network Connectivity Check

```
try:
    cmd = ["ping", "-c", "2", "google.com"]
    result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
    print("✅ Network: ONLINE" if result.returncode == 0 else "❌ Network: OFFLINE")
except subprocess.TimeoutExpired:
    print("⏰ Timeout — network unreachable")
```

Automates internet connectivity validation — useful for deployment or monitoring scripts.

3️⃣ Shutil Module

Purpose :

`shutil` provides **high-level file operations** like copy, move, archive, and disk usage monitoring.

Used for:

- File management
- Automated backups
- Directory organization

Example: File Operations

import shutil, os

# Copy file
shutil.copy2("scan_results.txt", "scan_results_backup.txt")

# Copy entire directory
shutil.copytree("security_scans", "backup_folder", dirs_exist_ok=True)

# Move or rename file
shutil.move("config.json", "configuration.json")

Ideal for automating **backup pipelines**, **report archiving**, and **configuration management**.

Example: Disk Usage and Archiving

```
# Disk usage
total, used, free = shutil.disk_usage(".")
print(f"Total: {total // (2**30)} GB | Free: {free // (2**30)} GB")

# Create a ZIP backup
shutil.make_archive("security_backup", "zip", "security_scans")
print("✅ Backup archive created: security_backup.zip")
```

Real-life Usage: Automatically archive scan results and system logs before cleanup or deployment.

Error Handling in OS-Level Operations

```
try:
    shutil.copy("nonexistent.txt", "backup/")
except FileNotFoundError as e:
    print(" File not found:", e)
```

🔹 Always include **try–except** to prevent automation script failure due to missing files or permissions.
🔹 Critical for **production-grade scripts** running in CI/CD or server environments.

Summary Table

Module	Core Functionality	Real-Life Use Case
os	File system and environment management	Log file handling, directory creation
subprocess	Execute shell/system commands	Network, process, or security checks
shutil	File copy, move, archive operations	Backup automation, system maintenance

---

Key Takeaways

- ➢ Automate OS-level administrative tasks efficiently
- ➢ Use `subprocess` for executing commands securely
- ➢ Implement `shutil` for backups and file maintenance
- ➢ Always apply **error handling** for production-grade reliability

---